

TECHNISCHE
UNIVERSITÄT
WIEN

C Programming

191.002 Operating Systems VU
2026W

Luca Di Stefano, Jaroslav Klapalek, Joel Klimont, Felix Resch,
Peter Puschner
2026-10-13

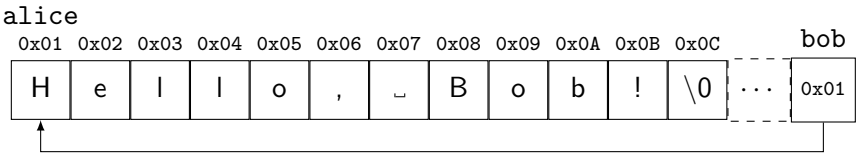
Pointers to const and const pointers

- `const int *p`: pointer to a constant integer
  - The integer value cannot be changed via `p`
  - `p` can be changed to point to another integer (even not `const`)
- `int * const p`: constant pointer to an integer
  - The integer value can be changed via `p`
  - `p` cannot be changed to point to another address
- Can be nested: `int * const * p`

  (Mutable pointer to a constant pointer to an integer) (right-to-left)

Good practice: Use `const` in function parameters whenever possible

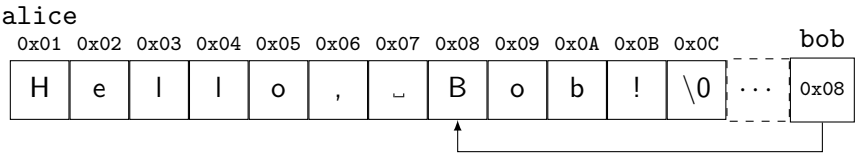
- Avoids accidental changes
- Guarantees to the caller that the argument won't be changed




```c
#include <stdio.h>
char alice[] = "Hello, Bob!";
char *bob = alice;
char *bob2 = &alice[0];      // Also ok
char *bob3 = &alice;         // Will trigger a warning

printf("%s\n", alice);       /* "Hello, Bob!" */
printf("%s\n", bob);         /* "Hello, Bob!" */

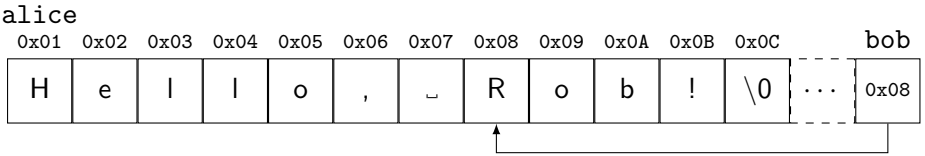
bob += 7;                    // Increment by 7 * sizeof(char) = 7 bytes
printf("%s\n", bob);         /* "Bob!" */
```




```c
#include <stdio.h>
char alice[] = "Hello, Bob!";
char *bob = alice;
char *bob2 = &alice[0];      // Also ok
char *bob3 = &alice;         // Will trigger a warning

printf("%s\n", alice);       /* "Hello, Bob!" */
printf("%s\n", bob);         /* "Hello, Bob!" */

bob += 7;                    // Increment by 7 * sizeof(char) = 7 bytes
printf("%s\n", bob);         /* "Bob!" */
```




```
*bob = 'R';  // <-- Characters are surrounded by single quotes!

printf("%s\n", bob);          /* "Rob!" */
printf("%s\n", alice);        /* "Hello, Rob!" */
```

This would happen even if `alice` was declared `const`!

(You only get a warning from the compiler, not an error.)

(Explicit) Casting

Tell the compiler to treat the result of an expression as a different type.

Floats to integers: cuts off the decimal part.

```
int pi = (int) 3.14;      // pi=3
```

Integers to integers:

- Casting to a larger type: keeps the value.
- Casting to a smaller type: cuts off higher bits.
- Casting between signed and unsigned may change the value.

```
int x = 2048 + 15;
printf("%#x\n", x);           // 0x80f  (= 2063)
printf("%#x\n", (char) x);    // 0xf    (= 15)
```

x

0	0	0x80	0xf

Casting pointers

Tell the compiler to treat a piece of memory as a different type.

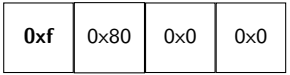
Behaviour may change depending on your machine!!!

```
int x = 2048 + 15;        // 2063, or 0x80f
char *y = (char*) &x;     // point 1st byte of x
printf("%#x\n", *y);      // ???
```

Little endian machine (Intel/AMD/Apple Silicon): `0xf`
Big endian machine (e.g. MIPS): `0`

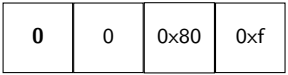
Little Endian:
lowest byte first

x



Big Endian:
highest byte first

x



Combines variables into a logical entity

- Contiguous block of memory
- Initialization: like arrays (C89) or by name (C99)
- Access elements with 'dot' (.)

```
1  struct account {
2    char username[32];
3    char password[32];
4    unsigned int uid;
5  };
6
7  struct account user1 = {"alice", "pass", 1};
8  struct account user2 = {.uid=0, .username="bob", .password="pass"};
9  user2.uid = user1.uid + 1;
```

Alignment

- Machine may require data to be aligned in memory (e.g., 4-byte words at addresses that is multiple of 4)
- Compiler inserts padding bytes to ensure alignment
- Use sizeof! Be careful with pointers within a struct!
- Compiler cannot reorder struct elements → do it manually!

```
1  struct mixed {
2    char  data1;
3    short data2;
4    int   data3;
5    char  data4;
6  }; /* 12 bytes */
```

```
1  struct mixed {
2    int   data3;
3    short data2;
4    char  data1;
5    char  data4;
6  }; /* 8 bytes */
```

Pointers to struct

`->` is a short notation for dereferencing (`*`) + access (`.`)

```
1  struct account {
2    char username[32];
3    char password[32];
4    unsigned int uid;
5  };
6
7  struct account user1 = { "alice", "pass", 1 };
8  struct account *p = &user1;
9
10 (*p).uid = 1;
11 p->uid   = 1; /* easier to read */
```

typedef

- User-defined types, e.g., `typedef unsigned int userId_t;`
- Very useful for structs
- Two syntaxes:

```c
struct account {
  char username[32];
  char password[32];
  unsigned int uid;
};
typedef struct account account_t;
```

```c
// Combined syntax (shorter)
typedef struct account {
  char username[32];
  char password[32];
  unsigned int uid;
} account_t;
```

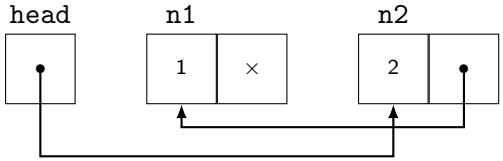
...then we can use `account_t` like any other type:

```c
account_t user1 = {"alice", "pass", 42};
```

Singly linked list

```c
typedef struct node {
  int data;
  struct node *next;
} node_t;
```

```c
// initially empty list
node_t *head = NULL;
node_t n1 = { .data = 1, .next = NULL };
node_t n2 = { .data = 2, .next = &n1 };
head = &n2;
```



(Assume one square = 4 bytes)

Function parameters


```
1 ret_type name(type1 arg1, type2 arg2, ..., typeN argN) {
2   /* code */
3 }
```

Attention: `int foo(void){...}` $\neq$ `int foo(){...}`

- `int foo(void);` does not accept any parameters
- `int foo();` accepts an arbitrary number of parameters
  - Think of `printf(...);`
- Remnant of K&R C (pre-ANSI)
- Deprecated in C99, so do not do this!

void

Introduced in ANSI-C as the “empty” data type

- return type for functions which return nothing
- empty parameter list for functions

`void*` = pointer with no data type

- can point to any data type
- pointer arithmetic is not allowed
- needs casting to another pointer type before dereferencing (* operator)

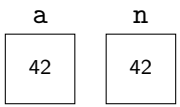
```
void *pv;              /* pointer to void */
int *pint;             /* pointer to int */
int i;

pv = (void *) &i;   /* casts are not necessary here, */
pint = (int *)pv;   /* but good style */
```

Call by value

- Function always works on a copy of the variable(s) in parameter list
- Arguments are expressions that are evaluated
- Resulting values are copied into the function's parameters
- Changes within the function do not affect the original variable

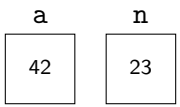
```
1  void foo(int n) {
2    n = 23;
3  }
4
5  int main(void) {
6    int a = 42;
7    foo(a);
8    /* the value of a is still 42 */
9    return 0;
10 }
```



Call by value

- Function always works on a copy of the variable(s) in parameter list
- Arguments are expressions that are evaluated
- Resulting values are copied into the function's parameters
- Changes within the function do not affect the original variable

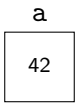
```
1  void foo(int n) {
2    n = 23;
3  }
4
5  int main(void) {
6    int a = 42;
7    foo(a);
8    /* the value of a is still 42 */
9    return 0;
10 }
```



Call by value

- Function always works on a copy of the variable(s) in parameter list
- Arguments are expressions that are evaluated
- Resulting values are copied into the function's parameters
- Changes within the function do not affect the original variable

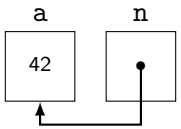
```
1  void foo(int n) {
2    n = 23;
3  }
4
5  int main(void) {
6    int a = 42;
7    foo(a);
8    /* the value of a is still 42 */
9    return 0;
10 }
```



Call by reference (using pointers)

- Write a function that takes a pointer as parameter
- The value of the pointer is copied (pass-by-value)
- The value pointed to by the pointer can be changed
- Note: arrays are always passed like this! (`arr` → `&arr[0]`)

```
1  void foo(int *n) {
2    *n = 23;
3  }
4
5  int main(void) {
6    int a = 42;
7    int *p = &a;
8    foo(p);
9    /* value of a is now 23 */
10   return 0;
11 }
```

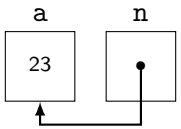


Call by reference (using pointers)

- Write a function that takes a pointer as parameter
- The value of the pointer is copied (pass-by-value)
- The value pointed to by the pointer can be changed
- Note: arrays are always passed like this! (`arr` → `&arr[0]`)

```c
void foo(int *n) {
  *n = 23;
}

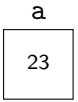
int main(void) {
  int a = 42;
  int *p = &a;
  foo(p);
  /* value of a is now 23 */
  return 0;
}
```



Call by reference (using pointers)

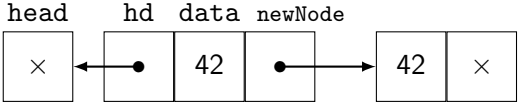
- Write a function that takes a pointer as parameter
- The value of the pointer is copied (pass-by-value)
- The value pointed to by the pointer can be changed
- Note: arrays are always passed like this! (`arr` → `&arr[0]`)

```
1  void foo(int *n) {
2    *n = 23;
3  }
4
5  int main(void) {
6    int a = 42;
7    int *p = &a;
8    foo(p);
9    /* value of a is now 23 */
10   return 0;
11 }
```



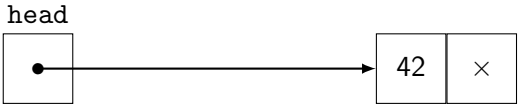
Singly linked list: add to front (push)

```
1  #include <stdlib.h> /* malloc */
2
3  void push(node_t * *const hd, const int data) {
4    node_t *newNode = malloc(sizeof(node_t));
5    newNode->data = data;
6    newNode->next = *hd;
7    *hd = newNode;
8  }
9  node_t *head = NULL;
10 push(&head, 42);
11 push(&head, 23);
```



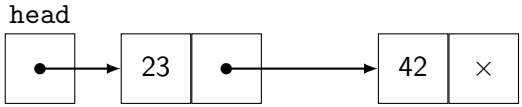
Singly linked list: add to front (push)

```
1  #include <stdlib.h> /* malloc */
2
3  void push(node_t * *const hd, const int data) {
4    node_t *newNode = malloc(sizeof(node_t));
5    newNode->data = data;
6    newNode->next = *hd;
7    *hd = newNode;
8  }
9  node_t *head = NULL;
10 push(&head, 42);
11 push(&head, 23);
```



Singly linked list: add to front (push)

```
1  #include <stdlib.h> /* malloc */
2
3  void push(node_t * *const hd, const int data) {
4    node_t *newNode = malloc(sizeof(node_t));
5    newNode->data = data;
6    newNode->next = *hd;
7    *hd = newNode;
8  }
9  node_t *head = NULL;
10 push(&head, 42);
11 push(&head, 23);
```

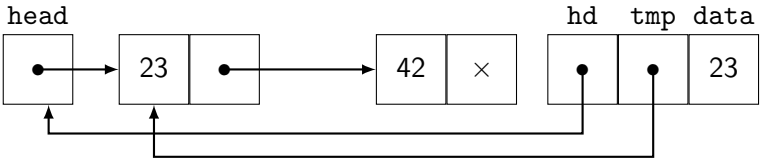


Caution

`malloc` returns `NULL` if no memory is available! Always check the returned pointer before using it

Remove and return the first element of the list

```
1  #include <stdlib.h> /* free */
2  int pop(node_t **hd) {
3    if(*hd == NULL) { return -1 }; /* list empty */
4    node_t *tmp = *hd; int data = tmp->data;
5    *hd = tmp->next;
6    free(tmp);
7    return data;
8  }
9  int val = pop(&head);
```



Singly linked list: pop

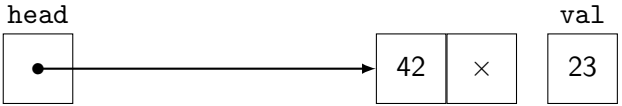
Remove and return the first element of the list

```
1  #include <stdlib.h> /* free */
2  int pop(node_t **hd) {
3    if(*hd == NULL) { return -1 }; /* list empty */
4    node_t *tmp = *hd; int data = tmp->data;
5    *hd = tmp->next;
6    free(tmp);
7    return data;
8  }
9  int val = pop(&head);
```

Caution

`free(p)` does not set `p` to `NULL`! (Do it by hand)

- Using pointer after *free* = undefined behavior
- Freeing a pointer twice = undefined behavior



Union

- Different interpretations of the same memory space
- `sizeof` returns the size of the biggest type
- Access elements with 'dot' (`.`), like structs

```
1  union number {
2    char  c_number;  /* 1 byte */
3    short s_number;  /* 2 bytes */
4  };
5  union number i;
6  i.c_number = 0x42;          /* sets 1 byte */
7  printf("%x", i.c_number); /* prints 42 */
8  i.s_number = 0x6548;       /* sets 2 bytes */
9  printf("%x", i.s_number); /* prints 6548 */
10 printf("%x", i.c_number); /* ?? undefined */
```

enum

Data type consisting of a collection of named integer constants

- By default values start at 0, but you can specify custom values
- If you use powers of 2, you can use bitwise operations on them
- You can `typedef` an enum like a struct

```
1  enum myBoolean {FALSE, TRUE}; // FALSE=0, TRUE=1
2  typedef enum permission {
3    READ = 1, WRITE = 2, EXEC = 4
4  } permission_t;
5
6  permission_t perm = READ | EXEC;
7  char canRead = perm & READ;      // true
8  char canWrite = perm & WRITE;    // false
```

Tagged unions

A struct containing: i) A union to store data; ii) A tag (enum) to indicate the correct interpretation of the union. Back to linked lists:

```
1  typedef enum types {
2    A_FLOAT, A_INT
3  } types_t;
4  typedef struct data {
5    types_t type;
6    union {
7      int i;
8      float f;
9    } value;
10 } data_t;
```

```
9  typedef struct node {
10   data_t data;
11   struct node *next;
12 } node_t;
13 //...
14 node_t *head = NULL;
15 data_t d1 = { A_INT, 42 };
16 data_t d2 = { .type=A_FLOAT, .value.f=3.14 };
17 push(&head, d1);
18 push(&head, d2);
```

Now you can store different data types in the same list!

Pointers to Functions

A pointer may point to a function too!

```
1  int add(int a, int b) { return a + b; }
2  int sub(int a, int b) { return a - b; }
3
4  int (*f)(int, int); // returnType (*name)(paramType1, paramType2, ...)
5  int ret;
6
7  f = &add;
8  ret = f(42,23);      /* ret == 65 */
9
10 f = sub;             /* f = &sub is better */
11 ret = (*f)(42,23);   /* ret == 19 */
```

Decoding C declarations: Useful links

- https://ptgmedia.pearsoncmg.com/images/9780131774292/samplepages/0131774298.pdf, p.76
- https://cdecl.org/

Modules

- Organize code across different files
- Increases readability, re-usability and maintainability
- Module is split into header (*.h) and source (*.c) files.

Header Files

- Contains prototypes and constants, but no definitions of functions
- Wrapped in include guards to avoid multiple inclusions (remember, this is all done as text replacement)
- `#include <file.h>` searches in library path
- `#include "file.h"` searches in local directory

```
1 /* linkedlist.h */
2 #ifndef LINKEDLIST_H /* include guard */
3 #define LINKEDLIST_H
4
5 typedef struct { ... } node_t;
6 void push(node_t * *const head, const int data);
7 int *pop(node_t * *const head);
8
9 #endif /* LINKEDLIST_H */
```

Source Files

- C files in which the functions are implemented
- Add `static` to function definition = available only in the current file

```c
/* linkedlist.c */
#include "linkedlist.h"

static int default(void) { return -1; }

void push(node_t **const head, const int *data) {
    ... /* See previous slides */
}

int pop(node_t **const head) {
  if (*head == NULL) { return default(); } /* list empty */
  ... /* See previous slides */
}
```

Use the module


```c
#include "linkedlist.h"

int main(void) {
  node_t *head = NULL;
  push(&head, 42);
  push(&head, 23);
  int val = pop(&head);

  return 0;
}
```

Compilation

Single-file compilation: concatenate all source files and compile

```
1 $ gcc -o prog prog.c linkedlist.c
```

Multi-file compilation: compile separate object files (*.o) and link them

```
1 $ gcc -c linkedlist.c # module                    -> linkedlist.o
2 $ gcc -c prog.c       # file that contains main() -> prog.o
3 $ gcc -o prog linkedlist.o prog.o
```

Single-file: Simple but recompiles everything, even if you changed one file

Multi-file: More complex but usually faster, more maintainable